

# Blue-Green Deployment

## on AWS Elastic Beanstalk with Terraform

Complete Project Documentation — Step-by-Step Guide + All Diagrams

|                   |                                               |
|-------------------|-----------------------------------------------|
| Author            | Ritik                                         |
| App Name          | my-app-bluegreen                              |
| Blue Environment  | my-app-bluegreen-blue (Production — App v1)   |
| Green Environment | my-app-bluegreen-green (Staging — App v2)     |
| Platform          | 64-bit Amazon Linux 2023 v6.11.0 — Node.js 20 |
| Instance Type     | t3.micro (Auto Scaling)                       |
| Swap Mechanism    | CNAME swap via AWS CLI — zero downtime        |
| IaC Tool          | Terraform   AWS Provider ~> 6.43.0            |
| AWS Region        | us-east-1                                     |
| Date              | 22-05-2026                                    |

### 1. Project Overview

This project implements a Blue-Green deployment strategy for a Node.js web application on AWS Elastic Beanstalk using Terraform. Two identical environments — Blue (production, running v1) and Green (staging, running v2) — run simultaneously. Traffic is served by Blue while Green is tested. When ready, a single PowerShell script swaps the DNS CNAMEs, making Green the new production instantly with zero downtime. Rollback is equally instant — just run the swap script again.

- Two live Elastic Beanstalk environments: Blue (v1, production) and Green (v2, staging)
- CNAME-based traffic swap — atomic DNS switch, no requests dropped
- Zero-downtime deployment — both environments run simultaneously
- Instant rollback — re-run swap script to go back to v1 in 1-2 minutes
- Separate IAM roles: EC2 instance role + Elastic Beanstalk service role
- Private S3 bucket stores both application ZIP files
- PowerShell scripts: package-apps.ps1 (zip apps) + swap-environments.ps1 (swap CNAMEs)
- Outputs: Blue and Green environment URLs and names

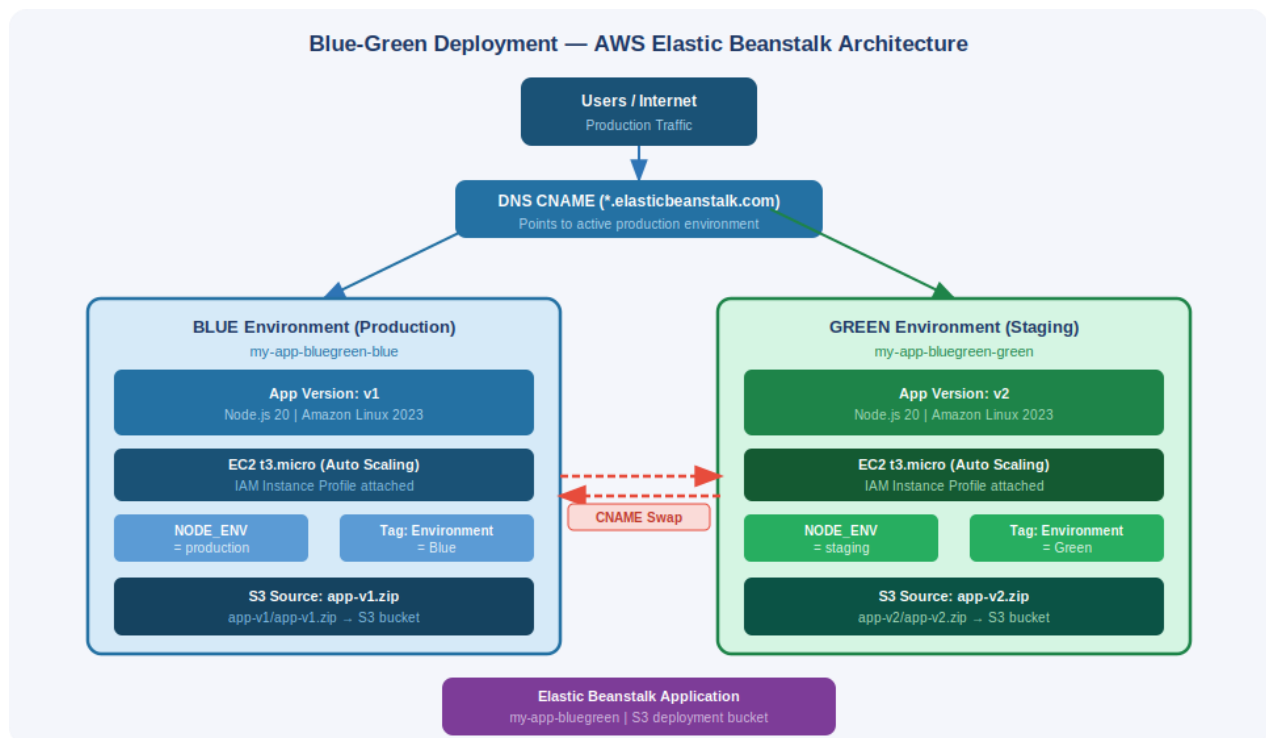
### 2. Project File Structure

### Project File Structure

```
blue-green-beanstalk/  
├─ main.tf      ─ EB app, S3 bucket, IAM EC2 + service roles  
├─ blue-environment.tf      ─ S3 object, app version v1, Blue EB env  
├─ green-environment.tf     ─ S3 object, app version v2, Green EB env  
├─ variables.tf    ─ aws_region, app_name, instance_type  
├─ outputs.tf      ─ blue/green URLs + environment names  
├─ providers.tf    ─ AWS provider ~> 6.43.0, region us-east-1  
├─ package-apps.ps1 ─ PowerShell: zips app-v1 and app-v2  
├─ swap-environments.ps1 ─ PowerShell: reads TF output, swaps CNAMEs  
├─ backend.tf      ─ (commented) S3 remote state  
├─ app-v1/app-v1.zip · app-v2/app-v2.zip · .terraform.lock.hcl · terraform.tfstate
```

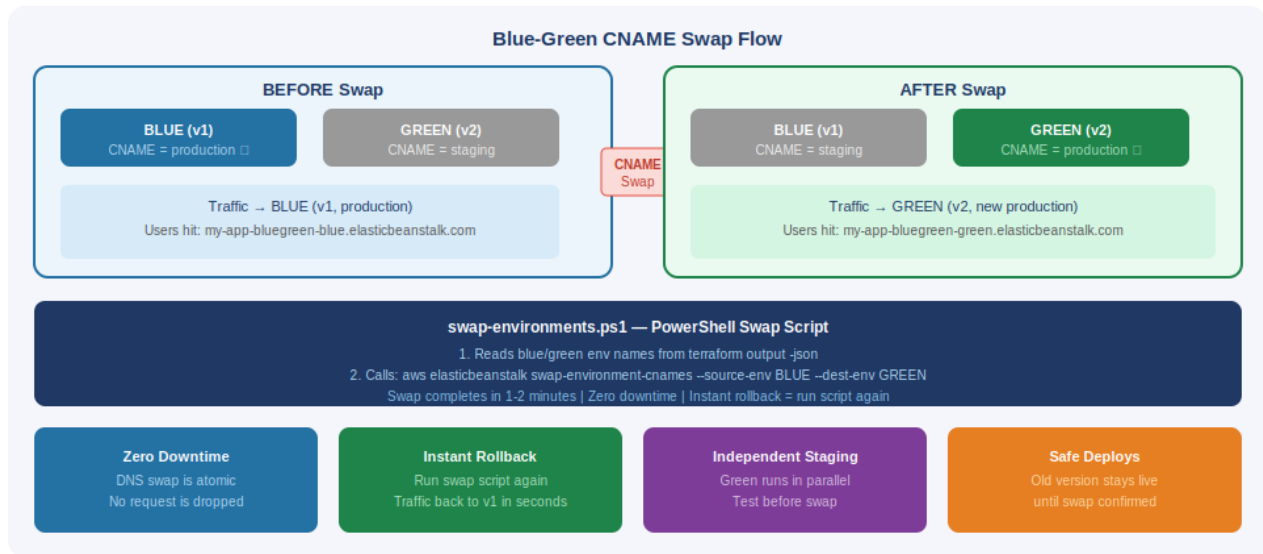
## 3. Architecture Diagram

The full infrastructure: both Elastic Beanstalk environments, their app versions, EC2 instances, S3 bucket, IAM roles, and the CNAME-based traffic routing between them.

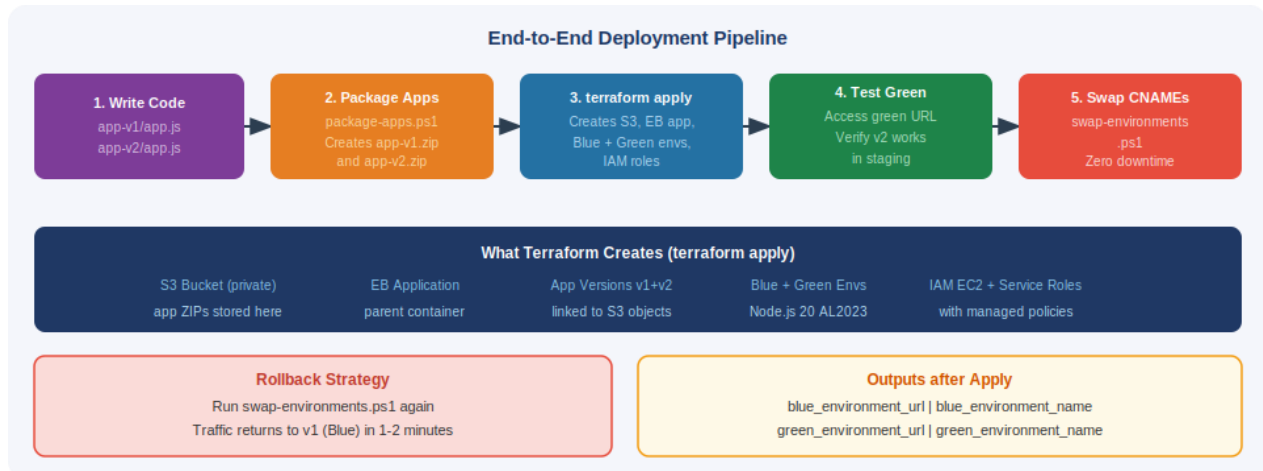


## 4. Blue-Green CNAME Swap Flow

Before the swap: Blue holds the production CNAME, Green is staging. After the swap: Green becomes production, Blue becomes staging. The swap is atomic — no traffic is lost.

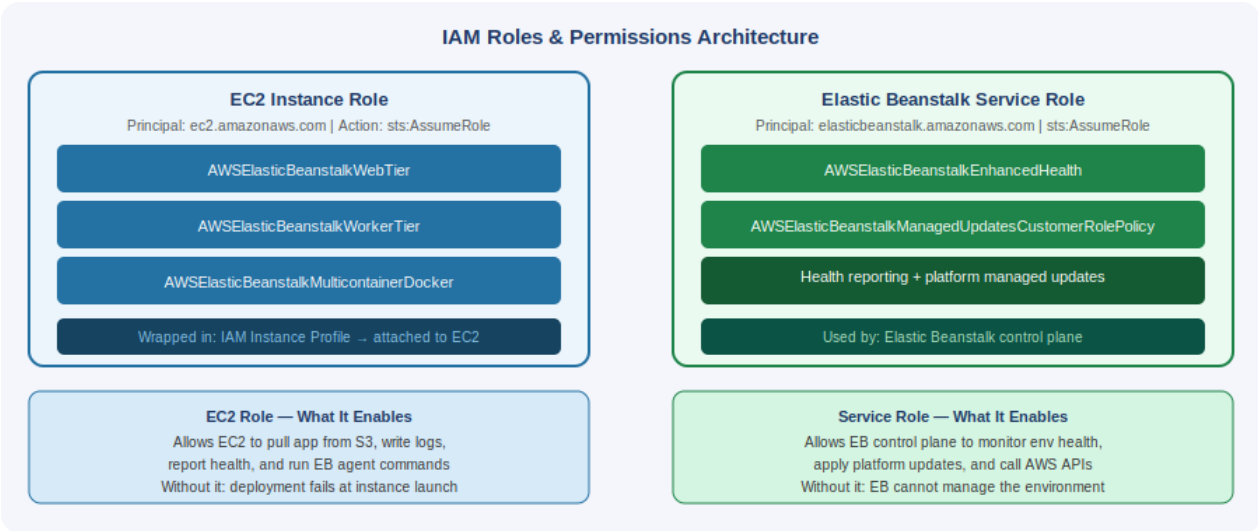


## 5. End-to-End Deployment Pipeline



## 6. IAM Roles & Permissions

Two distinct IAM roles are required for Elastic Beanstalk to function. The EC2 role is assumed by the instances themselves (to pull app ZIPs, write logs, etc.). The service role is assumed by the Elastic Beanstalk control plane (to manage environment health and updates).



## 7. Resources — What Each Does & Why

| Resource                                                    | What It Does                                          | Why It Is Needed                                                                                                                                            |
|-------------------------------------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>aws_elastic_beanstalk_application</code>              | Parent container for all EB environments and versions | Required before any environment or app version can be created. Acts as a namespace grouping all versions and environments for one application.              |
| <code>aws_s3_bucket (eb_bucket)</code>                      | Private S3 bucket storing both app ZIP files          | Elastic Beanstalk pulls application code from S3 — it cannot run code directly from local disk. The bucket is private (public access blocked) for security. |
| <code>aws_s3_bucket_public_access_block</code>              | Blocks all public access to the S3 bucket             | Prevents accidental exposure of application code. EB accesses the bucket via IAM roles, not public URLs — public access is never needed.                    |
| <code>aws_s3_object (blue_app)</code>                       | Uploads app-v1.zip to S3 with MD5 change detection    | app-v1.zip is the source for the Blue environment. etag = filemd5() detects code changes and triggers a re-upload + re-deployment automatically.            |
| <code>aws_s3_object (green_app)</code>                      | Uploads app-v2.zip to S3 with MD5 change detection    | app-v2.zip is the source for the Green environment. Same etag mechanism — Terraform re-deploys only when the ZIP content actually changes.                  |
| <code>aws_elastic_beanstalk_application_version (v1)</code> | Registers app-v1.zip as a named application           | EB needs a versioned snapshot of the code. The environment                                                                                                  |

|                                                             |                                                      |                                                                                                                                                          |
|-------------------------------------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                             | version                                              | references version v1 — this is what gets deployed to Blue EC2 instances.                                                                                |
| <code>aws_elastic_beanstalk_application_version (v2)</code> | Registers app-v2.zip as application version v2       | Same pattern for Green. v2 is the new code being staged. Keeping both versions registered allows rollback without re-uploading.                          |
| <code>aws_elastic_beanstalk_environment (blue)</code>       | Creates the Blue production EB environment           | Runs app v1 on Node.js 20/AL2023. <code>NODE_ENV=production</code> . Tagged as Blue. Receives live traffic before the swap.                              |
| <code>aws_elastic_beanstalk_environment (green)</code>      | Creates the Green staging EB environment             | Runs app v2 on Node.js 20/AL2023. <code>NODE_ENV=staging</code> . Tagged as Green. Tested in staging before the CNAME swap promotes it to production.    |
| <code>solution_stack_name</code>                            | Defines platform: Node.js 20 on Amazon Linux 2023    | Both environments must use the same platform stack for CNAME swap to be valid. Node.js 20 on AL2023 (v6.11.0) is the latest LTS stack.                   |
| <code>aws_iam_role (eb_ec2_role)</code>                     | IAM role assumed by EC2 instances in the environment | EC2 instances need permissions to download app ZIPs from S3, write CloudWatch logs, and communicate with EB APIs. Without this role, deployments fail.   |
| <code>aws_iam_role_policy_attachment (x3 EC2)</code>        | Attaches WebTier, WorkerTier, MulticontainerDocker   | These AWS-managed policies grant the EC2 role exactly the permissions EB needs — no over-permissioning. Separate attachments keep policies modular.      |
| <code>aws_iam_instance_profile</code>                       | Wraps EC2 role into an instance profile              | EC2 instances cannot directly assume IAM roles — they need an instance profile as the wrapper. EB attaches this profile to every EC2 it launches.        |
| <code>aws_iam_role (eb_service_role)</code>                 | IAM role assumed by EB control plane                 | Elastic Beanstalk itself (not the EC2 instances) needs permissions to: monitor environment health, perform platform updates, and call AWS service APIs.  |
| <code>aws_iam_role_policy_attachment (x2 Service)</code>    | Attaches EnhancedHealth + ManagedUpdates policies    | EnhancedHealth lets EB report detailed health metrics. ManagedUpdates allows EB to auto-apply platform patches. Without these, EB shows degraded health. |

|                                    |                                                      |                                                                                                                                                           |
|------------------------------------|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>package-apps.ps1</code>      | PowerShell: zips app.js + package.json per version   | Elastic Beanstalk expects a ZIP file. This script packages each app version (v1 and v2) into the correct ZIP format before terraform apply.               |
| <code>swap-environments.ps1</code> | PowerShell: reads TF output, calls EB CNAME swap API | Reads blue/green env names from terraform output -json, then calls aws elasticbeanstalk swap-environment-cnames. Swap takes 1-2 mins, zero downtime.      |
| <code>outputs.tf</code>            | Prints Blue/Green URLs and environment names         | Makes the environment endpoints immediately accessible after apply. The swap script also reads these outputs programmatically via terraform output -json. |

## 8. Step-by-Step Implementation Procedure

### Step 1 — Configure AWS Provider (providers.tf)

The AWS provider was configured for us-east-1 with version pinned to ~> 6.43.0. Version pinning prevents unexpected breaking changes from provider upgrades.

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws" version = "~> 6.43.0" }
  }
}

provider "aws" { region = "us-east-1" }
```

### Step 2 — Define Variables (variables.tf)

Three variables were defined: the AWS region, the application name (used to prefix all resource names), and the EC2 instance type. Using variables avoids hardcoding values and makes the configuration reusable.

```
variable "aws_region"    { default = "us-east-1" }
variable "app_name"      { default = "my-app-bluegreen" }
variable "instance_type" { default = "t3.micro" }
```

### Step 3 — Write Node.js Applications (app-v1 and app-v2)

Two separate Node.js application versions were created in separate directories: app-v1/ (current production code) and app-v2/ (new version to be deployed to Green). Each contains an app.js and package.json. The apps serve different responses so it is easy to verify which version is live after the swap.

```
app-v1/app.js      ← responds: 'Version 1 - Blue Environment'
app-v2/app.js      ← responds: 'Version 2 - Green Environment'
app-v1/package.json ← Node.js dependencies for v1
```

```
app-v2/package.json ← Node.js dependencies for v2
```

## Step 4 — Package Both Apps (package-apps.ps1)

A PowerShell script was written to compress each application directory into a ZIP file. Elastic Beanstalk requires a ZIP containing the app files — it cannot deploy a raw directory. The script removes any existing ZIP before creating a fresh one.

```
.\package-apps.ps1
# Creates: app-v1/app-v1.zip and app-v2/app-v2.zip
# Uses Compress-Archive to zip app.js + package.json per version
# Run this BEFORE terraform apply whenever code changes
```

## Step 5 — Create S3 Bucket & EB Application (main.tf)

The Elastic Beanstalk application (parent resource) and a private S3 bucket were created. All four public access block settings are enabled on the bucket — EB accesses it via IAM roles, not public URLs. The EB application acts as a namespace grouping all versions and environments.

```
resource "aws_elastic_beanstalk_application" "app" {
  name          = var.app_name
  description    = "Blue Green Deployment Demo"
}

resource "aws_s3_bucket" "eb_bucket" {
  bucket = "${var.app_name}-deployment-demo"
}

resource "aws_s3_bucket_public_access_block" "eb_bucket_block" {
  bucket                  = aws_s3_bucket.eb_bucket.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
```

## Step 6 — Create IAM EC2 Role + Instance Profile (main.tf)

An IAM role was created for EC2 instances with three AWS-managed policy attachments (WebTier, WorkerTier, MulticontainerDocker). These give EC2 instances the permissions needed to function inside Elastic Beanstalk. The role is wrapped in an IAM Instance Profile, which is the mechanism for attaching IAM roles to EC2 instances.

```
resource "aws_iam_role" "eb_ec2_role" {
  name = "${var.app_name}-ec2-role"
  assume_role_policy = jsonencode({
    Statement = [{ Effect = "Allow"
                    Principal = { Service = "ec2.amazonaws.com" }
                    Action     = "sts:AssumeRole" }]
  })
}

# Three policy attachments: WebTier, WorkerTier, MulticontainerDocker
resource "aws_iam_instance_profile" "eb_instance_profile" {
```

```

name = "${var.app_name}-instance-profile"
role = aws_iam_role.eb_ec2_role.name
}

```

## Step 7 — Create Elastic Beanstalk Service Role (main.tf)

A second IAM role was created for the Elastic Beanstalk service itself (not EC2). This role is assumed by elasticbeanstalk.amazonaws.com and allows EB to report enhanced health metrics and apply platform managed updates. Without this role, EB environments show degraded health status.

```

resource "aws_iam_role" "eb_service_role" {
  name = "${var.app_name}-service-role"
  assume_role_policy = jsonencode({
    Statement = [{ Effect = "Allow"
      Principal = { Service = "elasticbeanstalk.amazonaws.com" }
      Action    = "sts:AssumeRole" }]
  })
}
# Two attachments: EnhancedHealth + ManagedUpdatesCustomerRolePolicy

```

## Step 8 — Create Blue Environment (blue-environment.tf)

The Blue environment was created in three linked resources: (1) aws\_s3\_object uploads app-v1.zip to S3 with etag-based change detection, (2) aws\_elastic\_beanstalk\_application\_version registers it as version 'v1', and (3) aws\_elastic\_beanstalk\_environment creates the live environment pointing at v1 with production settings.

```

# 1. Upload ZIP to S3
resource "aws_s3_object" "blue_app" {
  bucket = aws_s3_bucket.eb_bucket.id
  key    = "app-v1.zip"
  source = "app-v1/app-v1.zip"
  etag   = filemd5("app-v1/app-v1.zip")
}

# 2. Register application version
resource "aws_elastic_beanstalk_application_version" "v1" {
  name          = "v1"
  application    = aws_elastic_beanstalk_application.app.name
  bucket        = aws_s3_bucket.eb_bucket.id
  key           = aws_s3_object.blue_app.key
}

# 3. Create Blue environment
resource "aws_elastic_beanstalk_environment" "blue" {
  name          = "${var.app_name}-blue"
  application    = aws_elastic_beanstalk_application.app.name
  solution_stack_name = "64bit Amazon Linux 2023 v6.11.0 running Node.js 20"
  version_label  = aws_elastic_beanstalk_application_version.v1.name
  # settings: InstanceType, IamInstanceProfile, ServiceRole, NODE_ENV=production
  tags = { Environment = "Blue" }
}

```



```
}
```

## Step 9 — Create Green Environment (green-environment.tf)

The Green environment mirrors the Blue setup but uses app-v2.zip and version v2. NODE\_ENV is set to 'staging' to indicate it is the pre-production environment. Both environments run simultaneously on the same platform stack, which is required for CNAME swap compatibility.

```
resource "aws_s3_object" "green_app" {
  bucket = aws_s3_bucket.eb_bucket.id
  key    = "app-v2.zip"
  source = "app-v2/app-v2.zip"
  etag   = filemd5("app-v2/app-v2.zip")
}

resource "aws_elastic_beanstalk_application_version" "v2" { name = "v2" ... }
resource "aws_elastic_beanstalk_environment" "green" {
  name          = "${var.app_name}-green"
  version_label = aws_elastic_beanstalk_application_version.v2.name
  # NODE_ENV = staging | tags = { Environment = "Green" }
}
```

## Step 10 — Define Outputs (outputs.tf)

Four outputs were defined: Blue and Green environment URLs (CNAMEs) and their names. The swap script reads the names programmatically using terraform output -json — these outputs are what make the automation work without hardcoding environment names.

```
output "blue_environment_url" { value = aws_elastic_beanstalk_environment.blue.cname }
output "green_environment_url" { value = aws_elastic_beanstalk_environment.green.cname }
output "blue_environment_name" { value = aws_elastic_beanstalk_environment.blue.name }
output "green_environment_name" { value = aws_elastic_beanstalk_environment.green.name }
```

## Step 11 — Package Apps and Deploy

```
.\package-apps.ps1      # Step 1: zip both app versions
terraform init           # Step 2: download provider
terraform validate       # Step 3: check syntax
terraform plan           # Step 4: preview resources
terraform apply          # Step 5: deploy everything
```

After apply, both environments are live. Blue serves production (v1), Green serves staging (v2). URLs are printed in the output.

## Step 12 — Test Green Environment

Before swapping, the Green environment URL was opened in a browser to verify v2 is running correctly in staging. This is the key advantage of Blue-Green — you test the new version in a real, live environment (identical to production) before exposing it to users.

```
# Open in browser to verify:
http://<green_url>.elasticbeanstalk.com → 'Version 2 - Green Environment'
```

## Step 13 — Swap CNAMEs (swap-environments.ps1)

Once Green is verified, the swap script is run. It automatically reads environment names from Terraform output, then calls the AWS CLI to swap the CNAMEs. The swap is atomic — the DNS pointer flips in 1-2 minutes with no dropped requests. Green is now production, Blue is now staging.

```
.\swap-environments.ps1

# Script internals:
# 1. $tfOutput = terraform output -json | ConvertFrom-Json
# 2. $BlueEnv = $tfOutput.blue_environment_name.value
# 3. $GreenEnv = $tfOutput.green_environment_name.value
# 4. aws elasticbeanstalk swap-environment-cnames \
#     --source-environment-name $BlueEnv \
#     --destination-environment-name $GreenEnv \
#     --region us-east-1
```

## Step 14 — Rollback (if needed)

If v2 has issues after the swap, rollback is instant. Run the same swap script again — it reverses the CNAME, sending traffic back to Blue (v1) in 1-2 minutes. No re-deployment, no code changes, no downtime.

```
.\swap-environments.ps1 # Run again → traffic returns to Blue (v1)
```

---

## 9. Key Design Concepts

| Concept                          | Explanation                                                                                                                                                                                                                                |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CNAME Swap                       | Both EB environments have their own CNAME (*.elasticbeanstalk.com). Swapping transfers the production CNAME to the other environment atomically — users are re-routed in DNS without any server restart or downtime.                       |
| <code>etag = filemd5()</code>    | Terraform computes the MD5 of the ZIP file and stores it as the S3 object's ETag. On every apply, if the file has changed (new code), Terraform detects the difference and re-uploads and re-deploys. If unchanged, it skips the upload.   |
| <code>version_label</code>       | EB environments reference application versions by label (v1, v2). This decouples the code from the environment — you can deploy v1 to Blue and v2 to Green independently, and swap the traffic layer without touching the code layer.      |
| <code>solution_stack_name</code> | Both environments must use the exact same platform string for CNAME swap to be valid. Mismatched platforms (e.g. Node 18 vs Node 20) would be rejected by the swap API.                                                                    |
| Two IAM Roles                    | EC2 role (assumed by instances): needs S3 read, CloudWatch write, EB agent permissions. Service role (assumed by EB control plane): needs health monitoring and update management permissions. Mixing these would violate least-privilege. |

|                           |                                                                                                                                                                                                       |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NODE_ENV per env</b>   | Blue sets NODE_ENV=production, Green sets NODE_ENV=staging. This allows the same application code to behave differently per environment (e.g. logging verbosity, feature flags) without code changes. |
| <b>Separate .tf files</b> | blue-environment.tf and green-environment.tf are separated to make it clear which resources belong to which environment. This also makes it easy to update one environment independently.             |

---

## 10. Commands Reference

Package applications:

```
.\package-apps.ps1
```

Initialize Terraform:

```
terraform init
```

Validate configuration:

```
terraform validate
```

Preview all changes:

```
terraform plan
```

Deploy infrastructure:

```
terraform apply
```

View environment URLs:

```
terraform output
```

Swap Blue ↔ Green:

```
.\swap-environments.ps1
```

Rollback (swap back):

```
.\swap-environments.ps1 # run again
```

Destroy all resources:

```
terraform destroy
```

---

## 11. Project Summary — Step by Step

### Everything done in this project — concise numbered summary

1. Configured the AWS provider (us-east-1, pinned to ~> 6.43.0) and defined three variables: aws\_region, app\_name, and instance\_type (t3.micro).
2. Wrote two separate Node.js app versions — app-v1/ (current production code) and app-v2/ (new version) — each with app.js and package.json serving distinct responses for easy verification.
3. Created package-apps.ps1: a PowerShell script that compresses each app directory into a ZIP file (app-v1.zip and app-v2.zip) ready for Elastic Beanstalk deployment.

4. Provisioned a private S3 bucket (all public access blocked) to store both ZIP files. Elastic Beanstalk requires application code to be fetched from S3.
5. Created the Elastic Beanstalk application resource (parent container) that groups all environments and versions under a single named application.
6. Created an IAM EC2 role with three AWS-managed policy attachments (WebTier, WorkerTier, MulticontainerDocker) and wrapped it in an IAM Instance Profile for attachment to EC2 instances.
7. Created an IAM Service Role (assumed by elasticbeanstalk.amazonaws.com) with EnhancedHealth and ManagedUpdates policies so EB can monitor and manage environment health.
8. Created the Blue environment (blue-environment.tf): uploaded app-v1.zip to S3 with MD5 change detection, registered it as application version v1, and launched the Blue EB environment with NODE\_ENV=production.
9. Created the Green environment (green-environment.tf): same pattern for app-v2.zip and version v2, with NODE\_ENV=staging — running simultaneously alongside Blue.
10. Defined four outputs: blue/green environment URLs (CNAMEs) and names — used for browser verification and programmatic reading by the swap script.
11. Ran package-apps.ps1 → terraform init → validate → plan → apply. Both environments deployed successfully with Blue serving production (v1) and Green serving staging (v2).
12. Tested Green environment in browser to verify v2 is running correctly in staging before any traffic is sent to it.
13. Created swap-environments.ps1: reads Blue and Green environment names from terraform output -json, then calls aws elasticbeanstalk swap-environment-cnames to atomically redirect all production traffic to Green (v2). Swap completes in 1-2 minutes with zero downtime.
14. Verified post-swap: production URL now returns v2 response. Rollback tested: re-running the swap script returned traffic to Blue (v1) in under 2 minutes.

**Result: A fully working Blue-Green deployment pipeline on AWS Elastic Beanstalk — two live environments running simultaneously, traffic swapped atomically via CNAME in 1-2 minutes with zero downtime, and instant rollback by re-running the swap script. Entire infrastructure provisioned with a single terraform apply.**